

第四次实验：两进程 GEMM Pipeline 与通信-计算重叠

2023302121009 刘鼎琨

一、实验源码

1. TODO 1:

```
186 void Rank0Pipeline(const Problem& problem, const std::vector<double>& b) {
187     int total_tiles = TileCount(problem);
188
189     std::vector<double> payload_a(ResultElements(problem));
190     std::vector<double> payload_b(ResultElements(problem));
191     std::vector<double>* current_payload = &payload_a;
192     std::vector<double>* next_payload = &payload_b;
193
194     std::vector<double> c_tile(problem.tile_rows * problem.n);
195     MPI_Request request = MPI_REQUEST_NULL;
196
197     for (int k = 0; k < total_tiles; ++k) {
198         int start_row = k * problem.tile_rows;
199         int rows = std::min(problem.tile_rows, problem.n - start_row);
200
201         ComputeTile(problem, b, start_row, rows, c_tile);
202
203         if (request != MPI_REQUEST_NULL) {
204             MPI_Wait(&request, MPI_STATUS_IGNORE);
205         }
206
207         PackResult(problem, start_row, rows, c_tile, *current_payload);
208
209         MPI_Isend(current_payload->data(), ResultElements(problem), MPI_DOUBLE,
210                 1, TAG_RESULT, MPI_COMM_WORLD, &request);
211
212         std::swap(current_payload, next_payload);
213     }
214
215     if (request != MPI_REQUEST_NULL) {
216         MPI_Wait(&request, MPI_STATUS_IGNORE);
217     }
218
219     SendStop();
220 }
```

图 1 TODO 1 代码

2. TODO 2:

```

222 void Rank1Pipeline(const Problem& problem, std::vector<double>& c) {
223     int total_received_rows = 0;
224     std::vector<double> payload(ResultElements(problem));
225
226     while (total_received_rows < problem.n) {
227         MPI_Status status;
228         MPI_Probe(0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
229
230         if (status.MPI_TAG == TAG_STOP) {
231             double stop[2];
232             MPI_Recv(stop, 2, MPI_DOUBLE, 0, TAG_STOP, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
233             break;
234         }
235
236         MPI_Recv(payload.data(), ResultElements(problem), MPI_DOUBLE, 0, TAG_RESULT,
237                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
238
239         UnpackResult(problem, payload, c);
240
241         total_received_rows += static_cast<int>(payload[1]);
242     }
243 }

```

图 2 TODO 2 代码

二、运行结果

由于我安装编译文件的路径不是默认路径, 提供的 bat 文件无法找到对应的 MPI 服务, 我选择用最基础的指令编译 cpp 文件。

```

~/Desktop $ g++ mpi_matrix_pipeline_starter.cpp -o mpi_matrix_pipeline_starter.exe -I"./extract_msmapi_sdk/msmpisdk_extract/PFiles/Microsoft SDKs/MPI/Include" -L"./extract_msmapi_sdk/msmpisdk_extract/PFiles/Microsoft SDKs/MPI/Lib/x64" -lmsmpi~/Desktop $ "D:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 2 mpi_matrix_pipeline_starter.exe blocking\
>
mode:          blocking
mpi_size:      2
matrix_size:   1024 x 1024
tile_rows:    64
tile_count:    16
elapsed_sec:   22.345
check:         matched
status:        PASS
~/Desktop $ "D:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 2 mpi_matrix_pipeline_starter.exe pipeline 16
mode:          pipeline
mpi_size:      2
matrix_size:   1024 x 1024
tile_rows:    16
tile_count:    64
elapsed_sec:   18.8526
check:         matched
status:        PASS

```

图 3 运行结果截图

三、加速比测试

```
~/Desktop $ "D:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 2 mpi
_matrix_pipeline_starter.exe pipeline 16
mode:          pipeline
mpi_size:      2
matrix_size:   1024 x 1024
tile_rows:    16
tile_count:    64
elapsed_sec:   18.8526
check:         matched
status:        PASS
~/Desktop $ "D:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 2 mpi
_matrix_pipeline_starter.exe pipeline 64
mode:          pipeline
mpi_size:      2
matrix_size:   1024 x 1024
tile_rows:    64
tile_count:    16
elapsed_sec:   17.7648
check:         matched
status:        PASS
~/Desktop $ "D:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 2 mpi
_matrix_pipeline_starter.exe pipeline 256
mode:          pipeline
mpi_size:      2
matrix_size:   1024 x 1024
tile_rows:    256
tile_count:    4
elapsed_sec:   28.878
check:         matched
status:        PASS
```

图 4 性能对比截图

通过调整参数形成测试结果，将其绘制为表 1：

表 1 不同 tile_rows 对应的耗时对比

tile_rows	tile_count	耗时(秒)
16	64	18.85
64	16	17.76
256	4	28.88

四、问题分析

Q1: MPI 非阻塞通信 API 与 MPI_Wait 的关系

MPI_Isend / MPI_Irecv: 仅负责向系统提交一个通信请求并立即返回，允许 CPU 继续执行后续代码。

MPI_Wait: 是一个阻塞调用，用于确认某个 Request 已经彻底完成。

在 MPI_Isend 发起后、MPI_Wait 返回前，该 Request 对应的 buffer（内存

缓冲区)处于“不安全”状态。如果你在此期间修改 `buffer` 里的数据, MPI 可能会读到错误的值; 如果复用该 `buffer` 进行下一次发送, 会导致数据覆盖风险。

Q2: 比较三组 `tile_rows` (16, 64, 256) 的性能。(tile_rows 不是越细越好)

如表 1。tile_rows=16 时: 虽然 overlap 机会多, 但频繁调用 `MPI_Isend` 和结果打包产生的系统开销占用了过多 CPU 时间。

tile_rows=256 时: 块数太少 (对于 1024 矩阵仅 4 块), 导致流水线级数不足, 计算和通信无法充分重叠。

Q3: 解释 Pipeline 不一定更快的原因

如果切片太细, 处理每一块所需的 MPI 协议头开销、上下文切换和内存拷贝时间会超过重叠节省的时间。如果网络带宽本身就是瓶颈, 无论怎么重叠, 总的通信时间是恒定的。一旦通信时间远大于计算时间, Pipeline 的优势就消失了。

另外, 当矩阵规模很小时, 单块计算瞬间完成, 无法有效掩盖通信耗时。计算任务和 MPI 的后台数据传输可能会竞争同一物理内存通道, 导致计算变慢。

Q4: 拓展加分题的思路或代码尝试。

```
67 void PipelineExtraCreditEntry() {
68     // 假设将 local_batch 分成 K 个 micro_batches
69     for (int k = 0; k < K; ++k) {
70         // 1. 计算当前 micro-batch 的部分梯度 (Partial GEMM)
71         ComputeMicroBatchWGrad(..., micro_batch_grad[k]);
72
73         // 2. 如果不是第一块, 可以先 Wait 上一块的 Iallreduce 是否完成
74         // if (k > 0) MPI_Wait(&request[k-1], ...);
75
76         // 3. 发起当前块的非阻塞同步
77         MPI_Iallreduce(micro_batch_grad[k].data(), reduced_grad[k].data(),
78                       count, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD, &request[k]);
79     }
80
81     // 最后统一 Wait 并累加结果
82     for (int k = 0; k < K; ++k) {
83         MPI_Wait(&request[k], MPI_STATUS_IGNORE);
84     }
85 }
```

图 5 拓展代码

思考: 为什么需要多套 `micro_batch_grad` 缓冲区?

大概率因为 `MPI_Iallreduce` 在 `Wait` 返回前, MPI 引擎可能仍在读取该内存, 此时严禁覆盖或修改它。